

CS184/284A Spring 2025 Homework 1 Write-Up

Names:

Link to webpage: <https://cal-cs184-student.github.io/hw-webpages-123bingxichan/hw1/index.html>

Link to GitHub repository: <https://github.com/cal-cs184-student/hw-webpages-123bingxichan>



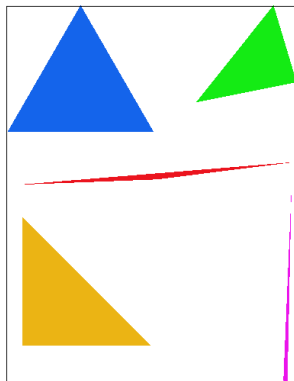
You can add images with captions!

Overview

In this homework, I implemented a rasterizer that can draw triangles, apply supersampling, apply transforms, use barycentric coordinates, and apply texture mapping. I also implemented mipmapping for texture mapping. The most interesting takeaways are first understanding everything from lecture better, but broke down what really happens when I type this message or play a video game into understandable chunks. Everytime something must be rendered, it uses some aspects of what I implemented in homeowrk, which is pretty cool!

Task 1: Drawing Single-Color Triangles

Given three 2D points, the goal is to understand which pixels are within those three points or not. To do this, I implemented the equation from lecture for `line_test` as a helper function, which then is called by another helper function inside, which then given a point and three points of a triangle, returns a boolean if that point is within the triangle or not by calling `line_test` 3 times. My algorithm cannot be worse than the one proposed because that's what I did. I can optimize later to only render points that will return true.

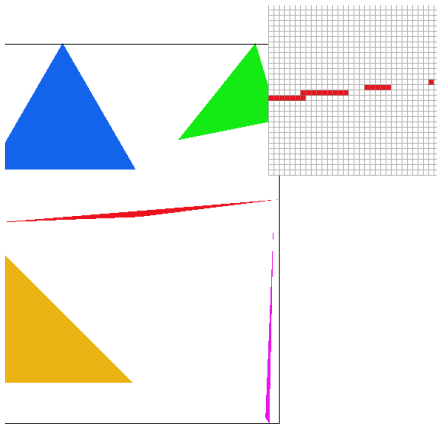


Triangles

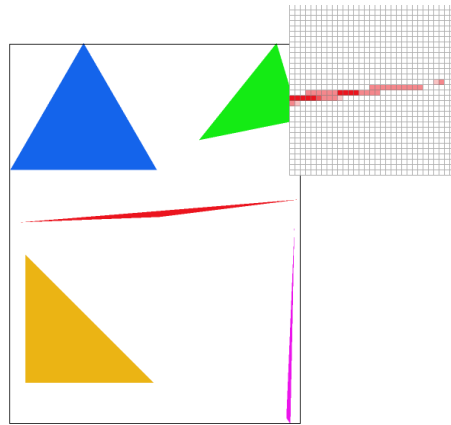
Task 2: Antialiasing by Supersampling

To implement supersampling, I updated the rasterize triangle function to loop through inner subsample points per pixel, then in `resolve_to_framebuffer` I took the average expression of each of these subpixels for the greater pixel. Supersampling is useful in eliminating 'the jaggies' at the edges of triangles, as when you increase resolution (sample rate) there are more possible pixels to fit into the triangle. The main alterations in the pipeline include increasing the `sample_buffer` size by the sample rate, then the subsampling in rasterize triangle. Using this subsampling, we can observe noticeably less aliasing as the sample rate (SR) increases, as by taking more samples we are ensuring a more accurate representation of the triangle.

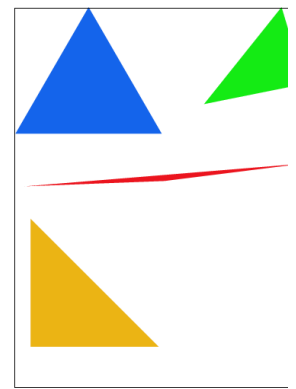
Here are comparisons at sample rates 1, 4, and 16:



Sample Rate 1



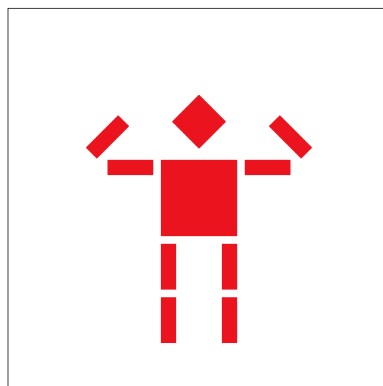
Sample Rate 4



Sample Rate 16

Task 3: Transforms

My robotman is doing a front double biceps pose to flex his biceps. All I had to do was bend his arms inwards, which included rotation and editing some scaling.

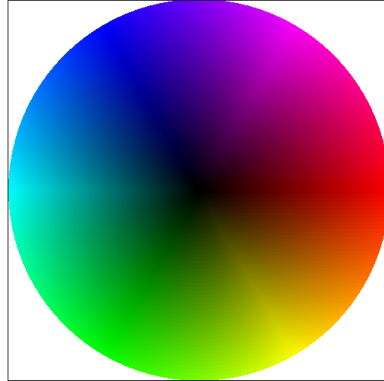


Front Double Bi

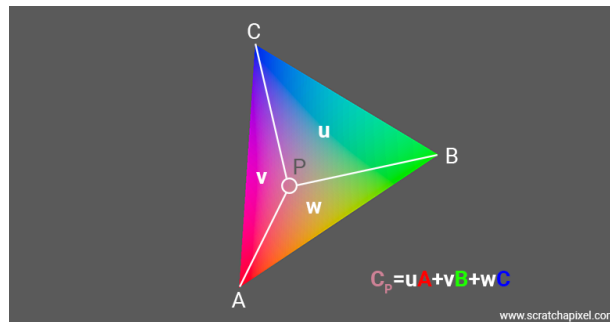
Task 4: Barycentric coordinates

My main confusion was understanding barycentric coordinates vs bilinear interpolation, as both achieve a similar goal of inputting a shape and trying

to figure out what proportion of each vertex should represent a point within the shape. Barycentric coordinates represent how far you are from each vertex, and thus tell you how much of each vertex to use for a given point. Bilinear interpolation does something very similar for squares, and helps you fill in the area in between each square. What is happening in the color wheel image is that we slice up that circle into many triangles, and calculate the barycentric coordinates for each triangle by inputting the vertex colors of each triangle, thus resulting in a gradient color wheel.



Color Wheel



Barycentric Coordinate Description, :

<https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-rendering-a-triangle/barycentric-coordinates.html>

Task 5: "Pixel sampling" for texture mapping

The problem we face is: given an image, apply a texture mapping to it. However, it is not guaranteed that the image and texture map will have the same dimensions, and therefore we run into the problem of an image coordinate which corresponds to a continuous coordinate in between two actual coordinates on the texture map. First, nearest sampling simply takes in a texture coordinate, finds the nearest point, and directly applies this value to the pixel. Bilinear interpolation tries to refine this by taking the four nearest points, then calculating the proportion of each vertex that should be represented in your coordinate. Going back to Task 4, imagine the left of the square being orange and red, and the right being blue and green — if your point was (0.75, 0.75) you would expect your point to be much more blue, which is what bilinear interpolation would return. In this image example, nearest sampling returns an aliased depiction of the campanile that is blocky and hard to parse, but with bilinear interpolation it becomes much smoother and less aliased.



Nearest Sampling



Bilinear Sampling

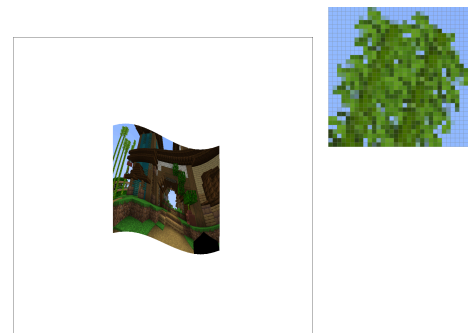
Task 6: "Level Sampling" with mipmaps for texture mapping

To me, mipmapping sounded most similar to an image pyramid in computer vision, where the idea is to convolve an image N times and yield a pyramid with sequentially lower resolution. The same idea applies here for level mapping — you want to know which layer of the mipmap should be applied to a specific part of the texture. Each level represents its own antialiasing, so the difficult part is trying to figure out which parts of the image should be at what level of the mipmap.

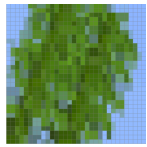
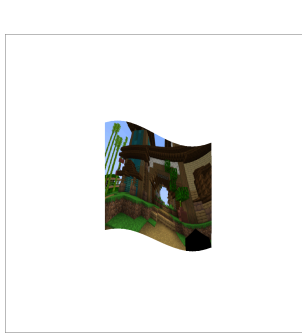
I thought Minecraft would be a good representation of different textures but it didn't work super well. In terms of speed and memory, I don't think I noticed any specific upsides or downsides. It just seems the more tools you apply, the more antialiasing you get, but also in turn more blurring. For my example I chose to focus on the bamboo because it was a bit far away and somewhat aliased. I noticed that bilinear pixel sampling made the image look the best, and any mipmapping caused too much blurring of the bamboo — kind of looking like I took my glasses off to see the image. I think the big differences come from the difference vector calculation; it could be that my mipmap is returning a very high value for each dx or dy , leading to a selection of a very large level and hence much more blurring.



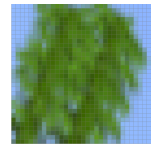
L0, Nearest Pixel (L0 PN)



L0, Bilinear Pixel (L0 PB)



Nearest Level, Nearest Pixel (LN PN)



Nearest Level, Bilinear Pixel (LN PB)